

REGISTRO DE BANCO DE DADOS - SEMANA 13

Código:

```
--
=====
=====
-- CONFIGURAÇÕES INICIAIS (Limpeza do ambiente)
--
=====
=====
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS clientes_premiados;
DROP TABLE IF EXISTS alertas;
DROP TABLE IF EXISTS auditoria_pedidos;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS clientes;

--
=====
=====
-- 1. CRIAÇÃO DAS TABELAS (DDL)
--
=====
=====

CREATE TABLE clientes (
  id_cliente  INTEGER PRIMARY KEY AUTOINCREMENT,
  nome        TEXT NOT NULL,
  email       TEXT NOT NULL UNIQUE,
  status_cliente TEXT DEFAULT 'Ativo',
  tipo_cliente TEXT DEFAULT 'online',
  data_cadastro TEXT DEFAULT (DATE('now'))
);

CREATE TABLE pedidos (
  id_pedido  INTEGER PRIMARY KEY AUTOINCREMENT,
  id_cliente INTEGER NOT NULL,
  data_pedido TEXT NOT NULL,
  status     TEXT NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

```

CREATE TABLE auditoria_pedidos (
    id_auditoria    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_pedido       INTEGER NOT NULL,
    status_antigo   TEXT,
    status_novo     TEXT,
    data_atualizacao TEXT NOT NULL
);

```

```

CREATE TABLE alertas (
    id_alerta    INTEGER PRIMARY KEY AUTOINCREMENT,
    mensagem     TEXT NOT NULL,
    data_alerta  TEXT NOT NULL
);

```

```

CREATE TABLE clientes_premiados (
    id_premiacao  INTEGER PRIMARY KEY AUTOINCREMENT,
    id_cliente    INTEGER NOT NULL,
    nome          TEXT NOT NULL,
    data_premiacao TEXT NOT NULL
);

```

```

--
=====
=====
-- 2. CRIAÇÃO DOS TRIGGERS (Automações e Regras de Negócio)
--
=====
=====

```

```

-- Trigger 1: Alerta automático ao inserir novo cliente
CREATE TRIGGER after_cliente_insert
AFTER INSERT ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO alertas (mensagem, data_alerta)
    VALUES ('Novo cliente cadastrado: ' || NEW.nome, DATETIME('now'));
END;

```

```

-- Trigger 2: Alerta para cancelamentos de pedidos
CREATE TRIGGER tg_alerta_pedido_cancelado
AFTER UPDATE OF status ON pedidos
WHEN NEW.status = 'Cancelado'

```

```
BEGIN
  INSERT INTO alertas (mensagem, data_alerta)
  VALUES ('ALERTA: O pedido ID ' || NEW.id_pedido || ' foi alterado para Cancelado.',
  DATETIME('now'));
END;
```

```
-- Trigger 3: Premiação automática se o status do cliente for alterado para VIP
CREATE TRIGGER tg_cliente_premiado
AFTER UPDATE OF status_cliente ON clientes
WHEN NEW.status_cliente = 'VIP'
BEGIN
  INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
  VALUES (NEW.id_cliente, NEW.nome, DATE('now'));
END;
```

```
-- Trigger 4: Auditoria de alteração de status (Apenas se houver mudança real)
CREATE TRIGGER after_pedido_update
AFTER UPDATE ON pedidos
FOR EACH ROW
WHEN OLD.status <> NEW.status
BEGIN
  INSERT INTO auditoria_pedidos (id_pedido, status_antigo, status_novo,
  data_atualizacao)
  VALUES (NEW.id_pedido, OLD.status, NEW.status, DATETIME('now'));
END;
```

```
-- Trigger 5: Deleção em cascata (Limpa os pedidos antes de apagar o cliente)
CREATE TRIGGER before_cliente_delete
BEFORE DELETE ON clientes
FOR EACH ROW
BEGIN
  DELETE FROM pedidos WHERE id_cliente = OLD.id_cliente;
END;
```

```
-- Trigger 6: Bloqueio de Segurança (Impede a exclusão de pedidos enviados)
CREATE TRIGGER before_pedido_delete
BEFORE DELETE ON pedidos
FOR EACH ROW
WHEN OLD.status = 'Enviado'
BEGIN
  SELECT RAISE(ABORT, 'Erro de Segurança: Não é permitido excluir pedidos já
  enviados.');
```

```
END;
```

```
--
=====
=====
-- 3. CARGA DE DADOS (DML)
--
=====
=====
```

```
-- Inserindo Clientes
INSERT INTO clientes (nome, email, status_cliente, tipo_cliente, data_cadastro) VALUES
('Lucas Silva', 'lucas@email.com', 'Ativo', 'online', '2026-05-20'),
('Ana Souza', 'ana@datassoft.com', 'Ativo', 'online', '2026-06-01'),
('Bruno Lima', 'bruno@datassoft.com', 'Ativo', 'online', '2026-06-02'),
('Carla Mendes', 'carla@datassoft.com', 'Ativo', 'presencial', '2026-06-03'),
('Diego Alves', 'diego@datassoft.com', 'Inativo', 'online', '2026-06-04');
```

```
-- Inserindo Pedidos
INSERT INTO pedidos (id_cliente, data_pedido, status) VALUES
(1, '2026-06-01', 'Pendente'),
(1, '2026-06-02', 'Processando'),
(1, '2026-06-03', 'Cancelado'),
(2, '2026-06-04', 'Enviado'),
(3, '2026-06-05', 'Pendente'),
(3, '2026-06-06', 'Cancelado'),
(4, '2023-01-10', 'Pendente');
```

```
-- Novo cliente (Para disparar o Trigger 1)
INSERT INTO clientes (nome, email, status_cliente, tipo_cliente)
VALUES ('Elaine Rocha', 'elaine@datassoft.com', 'Ativo', 'online');
```

```
--
=====
=====
-- 4. OPERAÇÕES E ATUALIZAÇÕES AVANÇADAS
--
=====
=====
```

```
-- Modificando status para gerar histórico na auditoria (Trigger 4)
UPDATE pedidos SET status = 'Processando' WHERE id_pedido = 1;
```

```

-- Premiação Manual via Query: Clientes online com 3 ou mais pedidos
INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
SELECT c.id_cliente, c.nome, DATETIME('now')
FROM clientes AS c
JOIN pedidos AS p ON c.id_cliente = p.id_cliente
WHERE c.tipo_cliente = 'online'
AND NOT EXISTS (
    SELECT 1 FROM clientes_premiados AS cp WHERE cp.id_cliente = c.id_cliente
)
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 3;

```

```

-- Promovendo Lucas a VIP para disparar a premiação automática (Trigger 3)
UPDATE clientes SET status_cliente = 'VIP' WHERE id_cliente = 1;

```

```

-- Atualização baseada em subquery (Clientes Inativos)
UPDATE pedidos
SET status = 'Revisar'
WHERE id_cliente IN (SELECT id_cliente FROM clientes WHERE status_cliente = 'Inativo');

```

```

-- Limpeza de dados (Deleta pedidos com mais de 2 anos que não foram enviados)
DELETE FROM pedidos
WHERE data_pedido < DATE('now', '-2 years') AND status <> 'Enviado';

```

```

-- Teste de exclusão em cascata (Trigger 5 - Apaga o cliente 3 e seus respectivos pedidos)
DELETE FROM clientes WHERE id_cliente = 3;

```

```

--

```

```

=====
=====

```

```

-- 5. CONSULTAS DE VERIFICAÇÃO (Para exibir os resultados na tela)

```

```

--

```

```

=====
=====

```

```

SELECT '--- CLIENTES ATUAIS ---' AS tabela;
SELECT * FROM clientes;

```

```

SELECT '--- PEDIDOS ATUAIS ---' AS tabela;
SELECT * FROM pedidos;

```

```

SELECT '--- AUDITORIA DE PEDIDOS ---' AS tabela;

```

```
SELECT '--- ALERTAS GERADOS ---' AS tabela;
SELECT * FROM alertas;
```

```
SELECT '--- CLIENTES PREMIADOS ---' AS tabela;
SELECT * FROM clientes_premiados;
```

Prints:

Output:

tabela

CLIENTES ATUAIS

id_cliente	nome	email	status_cliente	tipo_cliente	data_cadastro
1	Lucas Silva	lucas@email.com	VIP	online	2026-05-20
2	Ana Souza	ana@datassoft.com	Ativo	online	2026-06-01
4	Carla Mendes	carla@datassoft.com	Ativo	presencial	2026-06-03
5	Diego Alves	diego@datassoft.com	Inativo	online	2026-06-04
6	Elaine Rocha	elaine@datassoft.com	Ativo	online	2026-06-08

tabela

PEDIDOS ATUAIS

id_pedido	id_cliente	data_pedido	status
1	1	2026-06-01	Processando
2	1	2026-06-02	Processando
3	1	2026-06-03	Cancelado
4	2	2026-06-04	Enviado

tabela

AUDITORIA DE PEDIDOS

id_auditoria	id_pedido	status_antigo	status_novo	data_atualizacao
1	1	Pendente	Processando	2026-06-08 10:41:31

tabela

ALERTAS GERADOS

id_alerta	mensagem	data_alerta
1	Novo cliente cadastrado: Lucas Silva	2026-06-08 10:41:31
2	Novo cliente cadastrado: Ana Souza	2026-06-08 10:41:31
3	Novo cliente cadastrado: Bruno Lima	2026-06-08 10:41:31
4	Novo cliente cadastrado: Carla Mendes	2026-06-08 10:41:31
5	Novo cliente cadastrado: Diego Alves	2026-06-08 10:41:31
6	Novo cliente cadastrado: Elaine Rocha	2026-06-08 10:41:31

tabela

CLIENTES PREMIADOS

id_premiacao	id_cliente	nome	data_premiacao
1	1	Lucas Silva	2026-06-08 10:41:31
2	1	Lucas Silva	2026-06-08

Perguntas:

1. Qual é a diferença entre DDL e DML? Dê um exemplo de cada.

DDL (Data Definition Language - Linguagem de Definição de Dados): É usada para **criar, alterar ou excluir a estrutura** dos objetos no banco de dados (as "gavetas" onde os dados ficam). Os comandos modificam o esqueleto do banco.

Exemplo: CREATE TABLE (cria uma tabela) ou DROP TABLE (exclui uma tabela).

DML (Data Manipulation Language - Linguagem de Manipulação de Dados): É usada para **interagir com os dados** que estão guardados dentro dessas estruturas (o conteúdo das "gavetas").

Exemplo: INSERT INTO (insere um registro) ou UPDATE (altera as informações de um registro existente).

2. Por que uma chave primária é importante em uma tabela?

A Chave Primária (PRIMARY KEY) é o identificador exclusivo de uma linha em uma tabela. Ela é vital porque garante a **integridade dos dados** através de duas regras: **unicidade** (não podem existir duas linhas com o mesmo código) e **obrigatoriedade** (ela nunca pode ser nula). Sem ela, seria impossível distinguir ou atualizar com precisão dois clientes que tivessem exatamente o mesmo nome, por exemplo.

3. Para que serve uma chave estrangeira?

A Chave Estrangeira (FOREIGN KEY) serve para criar um **elo de ligação (relacionamento)** entre duas tabelas distintas e garantir a **integridade referencial**. Ela aponta para a Chave Primária de outra tabela e impede inconsistências no sistema — como permitir que um pedido seja cadastrado para um código de cliente que sequer existe no banco de dados.

4. O que o JOIN permite fazer?

O JOIN permite **juntar e consultar dados de duas ou mais tabelas na mesma resposta**, correlacionando as linhas através de seus campos em comum (geralmente ligando a Chave Estrangeira de uma tabela à Chave Primária de outra). Em vez de fazer várias consultas isoladas, o JOIN unifica as informações (como trazer o Nome do Cliente junto com a Data do Pedido dele).

5. Quando usamos GROUP BY e HAVING?

GROUP BY: É utilizado quando precisamos **agrupar linhas que possuem valores iguais** em determinadas colunas para realizar cálculos estatísticos ou resumos (funções de agregação como COUNT, SUM, AVG).

HAVING: É utilizado para **filtrar os resultados gerados por esse agrupamento**. O WHERE filtra linhas individuais *antes* do agrupamento; o HAVING filtra os blocos consolidados *depois* do agrupamento.

Exemplo: Agrupar pedidos por cliente (GROUP BY id_cliente) e mostrar apenas quem tem mais de 3 pedidos (HAVING COUNT(id_pedido) >= 3).

6. Por que o NOT EXISTS ajuda a evitar dados duplicados?

O NOT EXISTS funciona como um **teste de existência prévio**. Antes de inserir ou processar um dado, ele roda uma subconsulta rápida no banco para checar se aquele registro específico já foi salvo anteriormente. Se o registro já existir, a condição NOT EXISTS se torna falsa e o banco de dados ignora a nova inserção, blindando a tabela contra duplicidades.

7. O que uma trigger faz?

Uma Trigger (Gatilho) é um bloco de código SQL que fica "escutando" o banco de dados e é **executado de forma 100% automática** sempre que um evento de modificação de dados (INSERT, UPDATE ou DELETE) acontece em uma tabela específica. Ela serve para aplicar automações, logs e validações diretamente no motor do banco, sem depender do sistema ou do clique do usuário.

8. Por que registrar auditoria de alterações pode ser importante?

A auditoria é fundamental para a **segurança, conformidade (compliance) e depuração de erros**. Em sistemas corporativos, saber *quem* alterou um dado, *quando* alterou e *qual era o valor antigo* (como o histórico de status de um pedido ou preço de um produto) permite rastrear fraudes, corrigir falhas de integração humana ou de software e manter o histórico fiel da operação.

9. Por que bloquear a exclusão de pedidos enviados é uma boa prática?

Porque um pedido com status "Enviado" ou "Concluído" já disparou processos físicos, fiscais e financeiros reais (emissão de nota fiscal, saída do estoque, faturamento no cartão). Permitir que um usuário delete esse registro do banco de dados geraria um "furo" contábil e logístico irreversível, quebrando a consistência entre o que aconteceu no mundo físico e o que está registrado no sistema.

10. Explique com suas palavras a analogia da trigger como alarme automático.

Imagine que uma **Trigger** é como um **sensor de presença com alarme residencial**: Você não precisa ficar vigiando a casa ativamente nem precisa apertar um botão para o alarme tocar. O sensor fica instalado de forma invisível na janela (a tabela do banco). No momento exato em que alguém força a janela para entrar (um evento de INSERT, UPDATE ou

DELETE), o sensor detecta a ação sozinho e dispara a sirene imediatamente (executa o código SQL da Trigger). No caso do bloqueio de segurança com RAISE (ABORT), além de tocar o alarme, a trigger funciona como uma trava eletrônica que tranca as portas instantaneamente e impede o intruso de avançar.